

Xtquant实盘与Agent搭建实战



>> 今天的学习目标

Xtquant实盘与Agent搭建实战

- Xtquant实盘

CASE1: 连接账户与信息查询

CASE2: 下单与撤单

CASE3: 回调机制

CASE4: MiniQMTTrader 封装类

CASE5: 策略信号转实盘订单

- Nanobot实战

CASE: AI投研 (nanobot)

CASE: Content Builder (nanobot)

CASE: Text-to-SQL (nanobot)

CASE: Deep Research (nanobot)

CASE: nanobot下单

Nanobot SDK编程接口

- Skill Creator

biz-skill-creator 的三层架构设计

Skill Creator创建过程

生成质量评估: daily-morning-brief

Xtquant实盘

XtQuant使用

XtQuant 是 miniQMT 量化交易客户端的 Python API, 包含两大核心模块:

- **xttrader**: 交易模块, 包括: 下单、撤单、查询、回调 => 执行买卖、监控委托状态
- **xtdata**: 行情模块, 包括: K线、实时报价、历史数据 => 获取数据、计算指标

信号层 (xtdata + 策略计算) 和执行层 (xttrader + 下单) 分离, 各司其职。

CASE1：连接账户与信息查询

CASE1：连接账户与信息查询

场景：看看我的账户情况 => 有多少钱、持了什么股、今天委托了啥

核心路程：

创建 XtQuantTrader --> start() + connect() --> subscribe(账户) --> 查询资产/持仓/委托/成交 --> stop()

查询接口：

查询方法	返回内容	常用字段
query_stock_asset()	账户资产	total_asset, cash, market_value, frozen_cash
query_stock_positions()	股票持仓	stock_code, volume, can_use_volume, open_price
query_stock_orders()	当日委托	order_id, order_volume, traded_volume, order_status
query_stock_trades()	当日成交	traded_price, traded_volume, traded_amount

CASE1：连接账户与信息查询

```
from xtquant.xttrader import XtQuantTrader
from xtquant.xttype import StockAccount

# 创建交易实例 (session_id 区分不同策略)
trader = XtQuantTrader(QMT_PATH, session_id=100001)
trader.start()
trader.connect()

# 订阅账户
account = StockAccount(ACCOUNT_ID)
trader.subscribe(account)

# 查询
asset = trader.query_stock_asset(account)
print(f"总资产: {asset.total_asset:,.2f}")
print(f"可用资金: {asset.cash:,.2f}")
```

```
positions = trader.query_stock_positions(account)
for pos in positions:
    if pos.volume > 0:
        print(f"{pos.stock_code}: 持仓{pos.volume} 可用{pos.can_use_volume}")

trader.stop()
```

Thinking: session_id 的作用是什么?

同一个 miniQMT 客户端可被多个 Python 策略同时连接，每个策略必须使用不同的 session_id，否则会互相收到对方的回调推送。

CASE1：连接账户与信息查询

运行结果：

已连接 miniQMT，账户: cy，会话ID: 100001

--- 账户资产 ---

总资产:	0.00 元
可用资金:	0.00 元
持仓市值:	0.00 元
冻结资金:	0.00 元

--- 股票持仓 (2 只) ---

股票代码	持仓数量	可用数量	成本价	持仓市值
513100.SH	100	100	1.926	180.9
002432.SZ	3000	3000	49.517	278250.0

--- 当日委托 (6 笔) ---

委托编号	股票代码	买卖	委托数量	委托价格	成交数量	状态
672137217	513100.SH	买入	100	0.500	0	废单
672137218	513100.SH	买入	100	1.986	0	废单
672137219	513100.SH	买入	100	1.809	0	废单
672137220	513100.SH	买入	100	0.500	0	废单
672137221	513100.SH	买入	100	0.500	0	废单
672137222	513100.SH	买入	100	0.500	0	废单

--- 当日成交 ---

今日暂无成交

CASE2: 下单与撤单

CASE2: 下单与撤单

场景：买一手纳指ETF试试，然后把没成交的撤掉

下单代码

```
from xtquant import xtconstant
# 限价买入 -- 挂指定价格
order_id = trader.order_stock(
    account, "513100.SH", xtconstant.STOCK_BUY,
    100, xtconstant.FIX_PRICE, 1.00,
    '策略名', '备注'
)

# 市价买入 -- 沪市用 MARKET_SH_CONVERT_5_LIMIT
order_id = trader.order_stock(
    account, "513100.SH", xtconstant.STOCK_BUY,
    100, xtconstant.MARKET_SH_CONVERT_5_LIMIT, 0
)
```

撤单代码

单笔撤单

```
result = trader.cancel_order_stock(account, order_id)
```

批量撤单: 撤掉所有 "已报"/"部成" 的委托

```
orders = trader.query_stock_orders(account)
for order in orders:
    if order.order_status in (50, 55):
        trader.cancel_order_stock(account, order.order_id)
```

CASE2：下单与撤单

order_stock 参数

参数	说明	示例
account	StockAccount 账户对象	account
stock_code	股票代码	"513100.SH"
order_type	买卖方向	STOCK_BUY / STOCK_SELL
order_volume	委托数量 (100的整数倍)	100
price_type	报价类型	FIX_PRICE / MARKET_SH_...
price	限价填价格，市价填0	1.00 / 0

报价类型对照

类型	常量	说明
限价	FIX_PRICE	指定价格挂单
最新价	LATEST_PRICE	以当前最新价下单
沪市市价	MARKET_SH_CONVERT_5_LIMIT	五档即时成交，剩余转限价
深市市价	MARKET_SZ_CONVERT_5_CANCEL	五档即时成交，剩余撤销

CASE2：下单与撤单

委托状态码对照

状态码	含义	能否撤单
50	已报	可以
55	部成	可以
54	已撤	不需要
56	已成	无法撤
57	废单	无法撤

沪市 vs 深市: 市价委托类型不同:

- 沪市(.SH)用 MARKET_SH_CONVERT_5_LIMIT,
- 深市(.SZ)用 MARKET_SZ_CONVERT_5_CANCEL。

CASE2: 下单与撤单

运行结果:

已连接, 账户: cy

[步骤1] 限价买入 -- 挂 0.50 元低价

[限价买入] 513100.SH 100股 价格:0.5 委托编号:672137223

等待 2 秒...

[步骤2] 查看委托状态

513100.SH 编号:672137223 委托100股 状态:已报

[步骤3] 撤销委托 672137223

撤单请求已提交

[步骤4] 确认撤单结果

委托 672137223: 已撤

[附加] 批量撤销所有可撤委托

[批量撤单] 没有可撤的委托

CASE3：回调机制

CASE3：回调机制

场景：下单后怎么知道成交了没？

不用轮询，交易所主动告诉你

回调是事件驱动编程的核心: 你下单后不需要反复查询，交易所有任何变化都会通过回调主动推送给你。

回调方法	触发时机	参数
on_stock_order(order)	委托状态变化 (最常用)	XtOrder: stock_code, order_volume, order_status ...
on_stock_trade(trade)	有成交发生	XtTrade: traded_price, traded_volume, traded_amount ...
on_order_error(err)	委托被拒绝	error_id, error_msg
on_cancel_error(err)	撤单失败	error_id, error_msg
on_disconnected()	连接断开	无
on_order_stock_async_response	异步下单响应	order_id, seq

回调方法一览

CASE3：回调机制

```
class MyTraderCallback(XtQuantTraderCallback):

    def on_stock_order(self, order):
        # 每当委托状态变化: 未报->已报->已成/已撤
        direction = "买入" if order.order_type == 23 else "卖出"
        print(f"{order.stock_code} {direction} 状态:{order.order_status}")

    def on_stock_trade(self, trade):
        # 有成交时推送, 一笔委托可能分多次成交
        print(f"成交: {trade.traded_volume}股 价格:{trade.traded_price}")

    def on_order_error(self, order_error):
        print(f"委托失败: {order_error.error_msg}")
```

```
# 注册回调 (必须在 start() 之前)
callback = MyTraderCallback()
trader = XtQuantTrader(QMT_PATH, SESSION_ID)
trader.register_callback(callback)
trader.start()
trader.connect()
```

Thinking: 回调 vs 轮询的区别是什么?

- 轮询: 我每隔1秒问一次成交没
- 回调: 成交了交易所来告诉我

=> 回调更高效、更实时。

CASE3：回调机制

运行结果：

已连接，账户: cy

下限价单 (低价不会成交)，观察 on_stock_order 回调...

委托已提交，编号: 672137224

等待回调推送...

[09:31:02.145] [委托推送] 513100.SH 买入 委托100股 状态:未报 编号:672137224

[09:31:02.312] [委托推送] 513100.SH 买入 委托100股 状态:已报 编号:672137224

撤单，观察状态从 '已报' 变为 '已撤'...

[09:31:05.478] [委托推送] 513100.SH 买入 委托100股 状态:已报待撤 编号:672137224

[09:31:05.621] [委托推送] 513100.SH 买入 委托100股 状态:已撤 编号:672137224

回调事件日志 (共 4 条):

[09:31:02.145] [委托推送] 513100.SH 买入 委托100股 状态:未报 编号:672137224

[09:31:02.312] [委托推送] 513100.SH 买入 委托100股 状态:已报 编号:672137224

[09:31:05.478] [委托推送] 513100.SH 买入 委托100股 状态:已报待撤 编号:672137224

[09:31:05.621] [委托推送] 513100.SH 买入 委托100股 状态:已撤 编号:672137224

Thinking：观察回调顺序是怎样的？

下单后依次触发 未报 -> 已报 （两次 on_stock_order);

撤单后依次触发 已报待撤 -> 已撤。

=> 每次状态变化都会推送!

CASE4: MiniQMTTrader 封装类

CASE4: MiniQMTTrader 封装类

场景：能不能一行代码搞定？-- 把连接、下单、撤单、查询、回调全封装

定义类结构：

```
class MiniQMTTrader:
    # 连接管理
    connect() / disconnect()

    # 买卖 (自动区分沪深市价类型)
    buy(stock_code, volume, price=0)
    sell(stock_code, volume, price=0)

    # 撤单
    cancel(order_id) / cancel_all()
```

```
# 查询 (返回 dict/list)
query_asset() -> dict
query_positions() -> list[dict]
query_orders() -> list[dict]
query_trades() -> list[dict]

# 内置回调 + 风控
events -> list # 回调事件自动记录
```

CASE4: MiniQMTTrader 封装类

使用方式:

```
trader = MiniQMTTrader(qmt_path, account_id)
trader.connect()

trader.buy("513100.SH", 100)      # 市价买入一手
trader.buy("513100.SH", 100, price=1.0) # 限价买入
trader.sell("513100.SH", 100)     # 市价卖出

asset = trader.query_asset()
print(f"可用资金: {asset['cash']:, .2f}")

trader.cancel_all() # 撤掉所有可撤委托
trader.disconnect()
```

Thinking: 封装的价值是什么?

内部自动处理沪深市价类型切换、自动注册回调记录事件、内置持仓上限和单笔金额风控。后续都可以复用这个类（包括 nanobot 技能）

CASE4: MiniQMTTrader 封装类

运行结果:

[MiniQMTTrader] 连接成功 账户:cy

--- 账户资产 ---

total_asset: 278,430.90

cash: 0.00

market_value: 278,430.90

frozen_cash: 0.00

--- 股票持仓 ---

513100.SH: 持仓100 可用100 成本1.93

002432.SZ: 持仓3000 可用3000 成本49.52

--- 下单: 513100.SH ---

[买入] 513100.SH 100股 0.50 编号:672137225

--- 撤单 ---

[撤单] 编号:672137225 结果:成功

[撤单] 无可撤委托

--- 事件日志 (3 条) ---

[09:32:15] 委托推送: 513100.SH 买入 委托100股 已成交0股 状态:已报

[09:32:18] 委托推送: 513100.SH 买入 委托100股 已成交0股 状态:已报待撤

[09:32:18] 委托推送: 513100.SH 买入 委托100股 已成交0股 状态:已撤

[MiniQMTTrader] 已断开连接

CASE5：策略信号转实盘订单

CASE5：策略信号转实盘订单

场景：MACD 金叉自动买入 -- 让策略自己盯盘下单

=> 将策略信号自动转化为真实订单

架构：信号层 + 执行层分离

信号层 (xtdata + 策略计算)

- 获取实时行情 (xtdata)
- 计算技术指标 (MACD)
- 检测买卖信号 (金叉/死叉)
- 输出交易指令



执行层 (MiniQMTTrader)

- 接收交易指令
- 风控检查
- 下单到交易所
- 回调跟踪成交

CASE5：策略信号转实盘订单

两种运行模式：

1. 单次信号检查：

获取数据 -> 计算MACD -> 显示信号

=> 手动查看、定时任务

2. 实盘监控

持续循环: 获取数据 -> 检测信号 -> 自动下单

=> 策略自动执行

CASE5：策略信号转实盘订单

```
# 数据层: 使用 get_market_data_ex (推荐新版API)
```

```
data = xtdata.get_market_data_ex(
```

```
    field_list=['close'],
```

```
    stock_list=["513100.SH"],
```

```
    period='1d'
```

```
)
```

```
# 信号层: 检测 MACD 金叉/死叉
```

```
def check_signal(dif, dea):
```

```
    if prev_dif <= prev_dea and curr_dif > curr_dea:
```

```
        return 'buy' # 金叉
```

```
    if prev_dif >= prev_dea and curr_dif < curr_dea:
```

```
        return 'sell' # 死叉
```

```
# 执行层: 监控循环
```

```
while True:
```

```
    df = fetch_kline("513100.SH")
```

```
    dif, dea, _ = calc_macd(df['close'])
```

```
    signal = check_signal(dif, dea)
```

```
    if signal == 'buy':
```

```
        trader.buy("513100.SH", 100)
```

```
    elif signal == 'sell':
```

```
        trader.sell("513100.SH", 100)
```

```
    time.sleep(60)
```

CASE5：策略信号转实盘订单

运行结果：

请选择运行模式：

1. 单次信号检查

输入 1 或 2: 1

获取 513100.SH 历史行情...

获取 496 条K线

最近5个交易日：

日期	收盘价	DIF	DEA	MACD柱	信号
2026-04-03	1.731	-0.026861	-0.029912	0.006102	-
2026-04-07	1.732	-0.024486	-0.028827	0.008682	-
2026-04-08	1.833	-0.014289	-0.025919	0.023261	-
2026-04-09	1.805	-0.008370	-0.022409	0.028078	-
2026-04-10	1.809	-0.003319	-0.018591	0.030544	-

当前状态: DIF > DEA (多头)

最新信号: 无 (持续当前方向)

Thinking：通过MACD分析，当前行情如何？

DIF 从 -0.0269 上穿到 -0.0033，DEA 从 -0.0299 到 -0.0186，MACD柱持续放大(正)。

虽然 DIF 和 DEA 都为负(空头区域)，但 DIF > DEA 说明趋势在好转。尚未产生金叉信号，处于持续多头状态。

Summary

Thinking: 为什么信号层和执行层要分离?

- 策略逻辑可以独立修改和测试, 不影响下单代码
- 先用历史数据验证信号, 再接入实盘
- 执行层可以加风控检查, 不影响信号计算
- 信号层可以替换为任何策略 (MACD、均线、AI模型等)

Nanobot实战

Nanobot架构

Nanobot架构: Agent + Skills

Agent (AGENTS.md)

- 核心身份和行为准则
- 始终加载到 system prompt
- 协调各技能, 决定调用顺序



调用

Skills (SKILL.md + scripts/)

- 按需加载 (关键词匹配)
- 每个技能自包含脚本
- 通过 exec 工具执行

Nanobot技能

技能清单：

- stock-price: K线行情获取
- read-pdf: 研报/财报 PDF RAG
- financial-analysis: 财务指标分析
- sentiment-analysis: 舆情监控与情感分析
- investment-research: 五步法分析框架
- write-report: 研报撰写与导出
- compare-reports: 跨期/跨公司对比
- web-search: 联网搜索

Nanobot技能

TO DO: 理解Nanobot逻辑 (AI投研等) , 并添加交易相关技能

- 技能1: strategy-backtest (策略回测)

对指定股票运行 MACD / 双均线策略回测, 输出胜率、收益率、最大回撤、最新信号。

```
python scripts/run_backtest.py --code 513100.SH --strategy macd
```

- 技能2: trade-order (交易下单)

通过 miniQMT 执行买入、卖出、撤单, 查询账户资产和持仓。

```
python scripts/place_order.py --action buy --code 513100.SH --volume 100
```

```
python scripts/query_account.py --action positions
```

CASE: AI投研 (nanobot)

CASE: AI投研 (nanobot)

TO DO: 使用nanobot, 搭建AI投研

charles-nanobot/

config.json -- DashScope + Tavily + exec 配置

AGENTS.md -- Charles 核心身份 + 数据来源说明

agent.py -- 入口脚本 (交互模式 / 单次模式)

memory/

MEMORY.md -- 当前时间上下文 (运行时自动生成)

data/ -- (向量索引 + 财务数据)

skills/

investment-research/ -- 技能索引 (按场景选择技能)

write-report/ -- 五步法研报撰写 + 五种场景

web-search/ -- 联网搜索 + search_market.py 脚本

read-pdf/ -- PDF RAG 知识库 + 5个脚本

stock-price/ -- A股K线行情

financial-analysis/ -- 财务指标分析 + 同行对比

compare-reports/ -- 跨期/跨公司对比

sentiment-analysis/ -- 舆情监控 + 事件驱动

CASE: AI投研 (nanobot)

设计理念: 四层协同架构

AI投研 Agent 的核心设计是让 Nanobot 框架管理 Agent 生命周期, 让 SKILL.md 指导分析方法论:



CASE: AI投研 (nanobot)

AGENTS.md -- 身份定义 (始终加载)

Charles - AI 投研情报官

你是 Charles，一位专业的 AI 投研情报官。你为投资者提供深度研究分析服务。

核心方法论

采用国泰君安"五步法": 信息差 -> 逻辑差 -> 预期差 -> 催化剂 -> 结论+风险闭环

当用户要求写研报时，请先 read_file 读取 investment-research 技能获取详细指导。

数据来源

- 联网搜索: web_search 获取最新市场信息
- 本地知识库: exec 运行 PDF RAG 脚本
- 行情数据: exec 运行 K 线脚本
- 财务分析: exec 运行财务指标分析脚本

CASE: AI投研 (nanobot)

config.json -- 一处配置管全局

```
{
  "agents": { "defaults": {
    "model": "qwen-plus",
    "max_tool_iterations": 50,    // 最多工具调用轮数
    "context_window_tokens": 65536, // 上下文窗口
    "timezone": "Asia/Shanghai"
  }},
  "providers": { "dashscope": { "api_key": "" } },
  "tools": {
    "web": { "search": { "provider": "tavily" } },
    "exec": { "enable": true, "timeout": 120 }
  }
}
```

CASE: AI投研 (nanobot)

agent.py -- 入口脚本 (仅 ~100 行)

这个钩子只挂了一个点: `before_execute_tools` -- 在每次工具执行前, 把工具名和参数打印出来。

```
from nanobot.nanobot import Nanobot, _make_provider
from nanobot.config.loader import load_config
from nanobot.agent.hook import AgentHook, AgentHookContext
from nanobot.agent.loop import AgentLoop
from nanobot.bus.queue import MessageBus
```

```
class CharlesHook(AgentHook):
```

```
    """显示工具调用过程"""
```

```
    async def before_execute_tools(self, ctx: AgentHookContext):
```

```
        for tc in ctx.tool_calls:
```

```
            print(f" [{tc.name}] {str(tc.arguments)[:120]}")
```

```
def _inject_time_context():
```

```
    """将当前日期写入 memory/MEMORY.md, 自动注入 system prompt"""
```

```
    memory_file = WORKSPACE / "memory" / "MEMORY.md"
```

```
    today_str = datetime.now().strftime("%Y年%m月%d日")
```

```
memory_file.write_text(
```

```
    f"# 当前时间上下文\n\n今天是 {today_str}。 \n",
```

```
    encoding="utf-8",
```

```
)
```

```
def build_bot() -> Nanobot:
```

```
    config = load_config(WORKSPACE / "config.json")
```

```
    config.providers.dashscope.api_key = os.environ["DASHSCOPE_API_KEY"]
```

```
    config.tools.web.search.api_key = os.environ.get("TAVILY_API_KEY", "")
```

```
    config.agents.defaults.workspace = str(WORKSPACE)
```

```
    provider = _make_provider(config)
```

```
    loop = AgentLoop(
```

```
        bus=MessageBus(), provider=provider, workspace=WORKSPACE,
```

```
        model=config.agents.defaults.model,
```

```
        max_iterations=config.agents.defaults.max_tool_iterations,
```

agent.py

CASE: AI投研 (nanobot)

```
context_window_tokens=config.agents.defaults.context_window_tokens,
    web_config=config.tools.web, exec_config=config.tools.exec,
    timezone=config.agents.defaults.timezone,
)
_inject_time_context()
return Nanobot(loop)

# 交互式运行
bot = build_bot()
result = await bot.run("帮我撰写贵州茅台投研报告", hooks=[CharlesHook()])
print(result.content)
```

整个 agent.py 不包含任何投研逻辑或五步法代码。所有方法论指导写在 SKILL.md 中，通过 Skills 渐进式加载机制按需注入。

Agent 自主决定读取哪些技能、调用哪些工具、搜索哪些信息。

=> 更换分析方法论只需修改 SKILL.md 文件

CASE: AI投研 (nanobot)

运行结果:

=====

Charles 投研情报官 (nanobot 版)

模型: qwen-plus

技能: compare-reports | financial-analysis | investment-research | read-pdf | sentiment-analysis | stock-price | web-search | write-report

=====

你: 帮我撰写贵州茅台投研报告

[思考中...]

--- 第一阶段: 渐进式加载技能 ---

[read_file] skills/investment-research/SKILL.md ← 自主选择读取投研技能索引

--- 第二阶段: 迭代式信息收集 (6 轮搜索) ---

[web_search] 贵州茅台 600519 最新股价 2026年4月

[web_search] 贵州茅台 2025年年报 2026年一季报 财务数据

[web_search] 贵州茅台 2026年白酒行业分析 市场竞争格局

[web_search] 贵州茅台 2026年 分析师评级 投资建议

[web_search] 贵州茅台 2026年 批价 渠道改革 i茅台

--- 第三阶段: 本地财务数据分析 ---

[list_dir] data/financial_data/

[read_file] 600519_financial_abstract.csv

[read_file] 600519_income_statement.csv

[read_file] 600519_balance_sheet.csv

[read_file] 600519_cash_flow.csv

[exec] python skills/financial-analysis/scripts/ratio_analysis.py --stock 600519 -years 3

--- 第四阶段: 五步法分析 + 输出研报 ---

[write_file] reports/maotai_2026_q2_report.md ← 自动保存研报文件

Charles:

报告已生成! 严格遵循国泰君安"五步法"框架:

- 信息差: 淡季提价逻辑转变、渠道革命本质、现金流质量被低估
- 逻辑差: 市场对系列酒增长、毛利率可持续性、直营模式的三大误判
- 预期差: 修正盈利预测, 2026-2028年业绩增长路径
- 催化剂: 未来6-12个月关键事件时间表
- 结论: 强烈推荐, 目标价1850元 (+26.7%)

总计 15 次工具调用 | 耗时约 120s

CASE：AI投研（nanobot）

贵州茅台（600519）深度投研报告：市场化改革深化下的价值重估

报告日期：2026年4月6日 分析师：Charles AI投研情报官

执行摘要

贵州茅台正处于历史性转型的关键节点。在白酒行业整体承压（2025年Q3营收同比下滑18.4%）、中小酒企加速出清（2025年上半年规上酒企数量减少102家）的背景下，茅台通过**淡季精准提价+渠道结构重塑+i茅台直营放量**三重改革，成功实现从“配额套利”向“服务增值”的战略升级。2025年业绩逆势增长15.38%，ROE达36.02%，资产负债率仅19.04%，展现出顶级企业的经营韧性与财务健康度。当前股价1460元对应2026年PE约20.2X，低于历史中枢，具备显著安全边际。

核心投资建议：维持「强烈推荐」评级，目标价1850元（较现价+26.7%）

一、信息差：市场尚未充分认知的三大事实

1. 淡季提价的深层逻辑已发生根本性转变

历史提价（2003-2012黄金十年、2017年）：均在行业景气周期内进行，主要目的为稳定终端价格、缓解报表压力

2026年提价（2026年3月31日）：在行业深度调整期逆市操作，核心目标是**构建动态价格机制**——零售价随批价变化“随行就市”，终结长达8年的1499元官方指导价刚性束缚

效果验证：i茅台平台数据显示，调价后普飞批价稳定在1600-1750元区间，投机套利空间压缩40%以上

2. 渠道革命已实质性改变盈利模式

旧模式：“坐商”依赖配额差价（1169元出厂→1700元批价，毛利超500元/瓶）

新模式：推行“自售+经销+代售+寄售”四维协同，经销商转型为“服务商”，公司通过价差策略培育直营消费习惯

数据印证：2026年春节前i茅台普飞订单占比67%，贡献税前收入129亿元；42家直营门店出现抢购热潮

3. 现金流质量被严重低估

市场担忧经营现金流波动（2025年同比+38.85%至924.64亿元），实则源于财务公司存款结构调整

真实经营能力：每股经营现金流73.61元（+38.86%），远超行业均值；合同负债下降系主动为经销商减负的战略选择，非需求减弱信号

二、逻辑差：市场误判的三个关键环节

市场普遍认知	实际情况	市场影响/逻辑断层
“系列酒负增长=茅台整体承压”	系列酒仅为辅助业务，茅台酒主业依然坚挺	2025年茅台酒营收占比超85%，系列酒拖累整体增速
“高毛利率不可持续”	91.93%毛利率源于不可复制的护城河	品牌溢价（社交硬通货）、产区稀缺性
“直营扩张挤压经销商利润”	经销商转型后ROE提升	华源证券调研显示，转型为服务商后利润率提升

三、预期差：业绩确定性被显著低估

CASE：AI投研（nanobot）

盈利预测修正（单位：亿元）

年份	市场一致预期	本报告预测	差异	驱动因素
2026E	905.5	949.3	+4.8%	提价增厚+直营放量
2027E	949.3	1013.5	+6.8%	产能释放+产品结构优化
2028E	-	1125.0	-	"双百亿"产品格局成型

核心支撑逻辑：

提价弹性：经销渠道出厂价提升100元/瓶，按1亿瓶销量测算，直接增厚营收100亿元，贡献净利润约35亿元

直营红利：i茅台平台用户突破1400万，客单价提升至1499元，直营占比每提升1个百分点，ROE提升0.8个百分点

产品矩阵：2026年新增500ml茅台经典系列，定位大众消费，填补1000-1500元价格带空白

四、催化剂：未来6-12个月关键事件时间表

时间	事件	影响
2026年4月25日	2026年一季报披露	验证提价初期效果，预计营收/净利润增速超预期
2026年5月	i茅台平台接入微信小程序	用户触达效率提升，预计月活用户增长30%
2026年6月	《2026年贵州茅台酒市场化运营方案》细则落地	渠道费用政策明确，经销商转型加速
2026年9月	2026年中秋旺季动销数据	验证“金字塔”产品结构成效，批价稳定性关键
2026年12月	新一轮回购计划执行完毕	体现管理层信心，强化股东回报确定性

五、结论与风险提示

投资结论

贵州茅台已超越传统白酒企业范畴，进化为以**数字化为引擎、以消费者为中心、以市场化为准则**的高端消费品平台。其核心价值体现在：

确定性：950亿元净利润目标完成概率超90%，长期年化10%增长可期

稀缺性：全球烈酒领域唯一兼具文化符号、社交货币、金融属性的超级品牌

成长性：i茅台平台打开C端增量空间，直营占比提升带来ROE持续上行

目标价1850元计算依据：

2026年EPS 75.8元 × 24.4X PE（历史中枢+改革溢价） = 1850元

对应2026年PE 24.4X，较当前20.2X存在20.8%估值修复空间

风险提示

- 宏观经济超预期下行：**若GDP增速跌破4.5%，高端消费场景可能持续萎缩
- 政策监管风险：**禁酒令等政策若升级，可能影响商务宴请等核心消费场景
- 渠道改革不及预期：**经销商转型进度缓慢可能导致直营放量受阻
- 国际竞争加剧：**进口烈酒关税下调可能冲击高端价格带

附录：关键财务指标速览（2025年报）

营业总收入：1741.44亿元（+15.66% YoY）

归母净利润：862.28亿元（+15.38% YoY）

净资产收益率（ROE）：36.02%（+5.35pp YoY）

毛利率：91.93%（行业第一，稳居90%+）

资产负债率：19.04%（财务结构极度稳健）

Summary

- 渐进式加载生效

Agent 启动后第一个动作就是 read_file 读取 SKILL.md，而非一开始加载全部 8 个技能。

- 自主规划能力

Agent 自主将任务分解为：搜索 -> 读取本地数据 -> 运行分析脚本 -> 撰写报告

四个阶段，共 15 次工具调用。

- 多来源数据融合

结合了 web_search（联网实时信息）+ read_file（本地财务CSV）+ exec（运行分析脚本）

- 自动保存

Agent 主动使用 write_file 将研报保存为 Markdown 文件。



CASE: Content Builder (nanobot)

CASE: Content Builder (nanobot)

项目结构

nanobot-examples/content-builder/

config.json -- qwen-plus + tavily

AGENTS.md -- Brand voice + Content standards

agent.py -- 入口脚本

skills/blog-post/

SKILL.md -- 博客写作 4 步 workflow

不需要自定义工具。

内置的 web_search (研究素材) + write_file (保存文章) + Skills (写作流程指导) 即可

CASE: Content Builder (nanobot)

Content Builder Agent (nanobot)

Task: Write a short blog post about AI agents

```
>> write_file: {'path': 'ai-agents-blog-post.md', ...}
```

Agent Output:

```
# What Are AI Agents? (And Why They're More Than Just Chatbots)
```

Forget everything you thought you knew about AI assistants.

Today's AI agents aren't just conversation partners -- they're autonomous problem-solvers that can plan, reason, and act across multiple tools.

Beyond Simple Queries

Traditional chatbots respond to prompts. AI agents, however, break down complex goals into steps: research market data, analyze financial reports, draft an email, and even schedule follow-ups -- all without constant human direction.

Real-World Impact

Companies use AI agents for customer support escalation routing, automated code testing, and personalized learning paths. Unlike static scripts, agents adapt to new information and learn from outcomes.

The key difference? Agents do, while chatbots respond.

exit_code: 0 | elapsed: 47s

CASE: Text-to-SQL (nanobot)

CASE: Text-to-SQL (nanobot)

项目结构

nanobot-examples/text-to-sql/

config.json -- qwen-plus, exec 等

AGENTS.md -- SQL 查询规则

agent.py -- 入口 + QueryDBTool 自定义工具

chinook.db -- 自动下载的示例数据库

skills/

schema-exploration/ -- 数据库结构探索技能

query-writing/ -- SQL 编写与错误恢复技能



Text-to-SQL Agent (nanobot)

Question: How many customers are from Canada?

Answer: There are 8 customers from Canada in the database.

exit_code: 0 | elapsed: 35s

自定义 QueryDBTool 继承 Tool 基类

QueryDBTool: 自定义 SQL 查询工具

QueryDBTool -- 自定义 SQL 查询工具

```
class QueryDBTool(Tool):
    """SQL 查询工具 -- 在 Chinook 数据库上执行只读 SQL"""
    def __init__(self, db_path: Path):
        self._db_path = db_path

    @property
    def name(self) -> str:
        return "query_db"

    @property
    def description(self) -> str:
        return (
            "Execute a read-only SQL query against the Chinook database. "
            "Returns query results as formatted text. Only SELECT and "
            "PRAGMA statements are allowed."
        )
```

```
@property
def parameters(self) -> dict[str, Any]:
    return {
        "type": "object",
        "properties": {
            "sql": {
                "type": "string",
                "description": "The SQL query to execute (SELECT only)"
            }
        },
        "required": ["sql"]
    }

@property
def read_only(self) -> bool:
    return True
```

test-to-sql/agent.py

QueryDBTool: 自定义 SQL 查询工具

```
async def execute(self, **kwargs: Any) -> str:
    sql = kwargs.get("sql", "").strip()
    if not sql:
        return "Error: empty SQL query"

    upper = sql.upper().lstrip()
    if not (upper.startswith("SELECT") or upper.startswith("PRAGMA")):
        return "Error: only SELECT and PRAGMA statements are allowed"

    try:
        conn = sqlite3.connect(str(self._db_path))
        cursor = conn.cursor()
        cursor.execute(sql)
        columns = [desc[0] for desc in cursor.description] if
cursor.description else []
        rows = cursor.fetchall()
        conn.close()
```

```
if not rows:
    return "Query returned 0 rows."

    lines = [" | ".join(columns)]
    lines.append("-" * len(lines[0]))
    for row in rows[:50]:
        lines.append(" | ".join(str(v) for v in row))
    if len(rows) > 50:
        lines.append(f"... ({len(rows)} total rows, showing first 50)")

    return "\n".join(lines)
except Exception as e:
    return f"SQL Error: {e}"
```

QueryDBTool: 自定义 SQL 查询工具

逻辑分四层:

1) 工具元数据 (模型看到的 说明书)

```
name: "query_db"          → 模型调用时写 query_db(sql="...")
description: "Execute a read-only SQL..." → 告诉模型这个工具能做什么
parameters: { sql: string } → 只接受一个参数: SQL 语句
read_only: True           → 标记为只读工具
```

这些 property 会被框架的 ToolRegistry 收集, 转换成 LLM function calling 的 schema 发给模型。

2) 安全校验 (代码级硬限制)

```
upper = sql.upper().lstrip()
if not (upper.startswith("SELECT") or upper.startswith("PRAGMA")):
    return "Error: only SELECT and PRAGMA statements are allowed"
```

只允许两种语句:

SELECT -- 查数据

PRAGMA -- 查元信息 (表结构、列信息等)

DELETE、DROP、INSERT、UPDATE 全部被拦截。

QueryDBTool: 自定义 SQL 查询工具

3) 执行 SQL 并格式化结果

```
conn = sqlite3.connect(str(self._db_path)) # 打开数据库
cursor.execute(sql) # 执行 SQL
columns = [desc[0] for desc in cursor.description] # 取列名
rows = cursor.fetchall() # 取所有行
```

返回格式是文本表格，方便模型阅读：

```
CustomerId | FirstName | LastName | Country
```

```
-----
```

```
3 | Francois | Tremblay | Canada
```

```
14 | Mark | Philips | Canada
```

```
...
```

4) 安全限制：最多返回 50 行

```
for row in rows[:50]:
    ...
if len(rows) > 50:
    lines.append(f"... ({len(rows)} total rows, showing first 50)")
```

防止大表查询把模型的上下文窗口撑爆。

build_bot: 组装整个 Agent

build_bot: 组装整个 Agent

```
def build_bot() -> Nanobot:

    dashscope_key = os.environ.get("DASHSCOPE_API_KEY", "")
    if not dashscope_key:
        print("[Error] DASHSCOPE_API_KEY not set")
        sys.exit(1)

    config = load_config(WORKSPACE / "config.json")
    config.providers.dashscope.api_key = dashscope_key
    config.agents.defaults.workspace = str(WORKSPACE)

    provider = _make_provider(config)
    defaults = config.agents.defaults

    loop = AgentLoop(
        bus=MessageBus(),
```

```
        provider=provider,
        workspace=WORKSPACE,
        model=defaults.model,
        max_iterations=defaults.max_tool_iterations,
        context_window_tokens=defaults.context_window_tokens,
        max_tool_result_chars=defaults.max_tool_result_chars,
        web_config=config.tools.web,
        exec_config=config.tools.exec,
        restrict_to_workspace=False,
        timezone=defaults.timezone,
    )

    # 注册自定义 SQL 查询工具
    loop.tools.register(QueryDBTool(DB_PATH))

    return Nanobot(loop)
```

build_bot: 组装整个 Agent

```
build_bot()
```

```
|
```

```
└─ (1) 读取 API Key
```

```
|   dashscope_key = os.environ.get("DASHSCOPE_API_KEY")
```

```
|   没有就退出
```

```
|
```

```
└─ (2) 加载配置
```

```
|   config = load_config("config.json")
```

```
|   └─ model: "qwen-plus"
```

```
|   └─ max_tool_iterations: 20
```

```
|   └─ web: { enable: false }
```

```
|   └─ exec: { enable: true, timeout: 30 }
```

```
|
```

```
└─ (3) 创建 LLM Provider
```

```
|   provider = _make_provider(config)
```

```
|   → 连接通义千问 API 的适配器
```

```
└─ (4) 创建 AgentLoop (核心引擎)
```

```
|   AgentLoop(
```

```
|       bus      → 消息总线 (子 agent 通信用)
```

```
|       provider → LLM 适配器
```

```
|       workspace → 工作目录 (Skills/AGENTS.md 从这里加载)
```

```
|       model    → "qwen-plus"
```

```
|       max_iterations → 20 (最多调 20 轮工具)
```

```
|       web_config → 关闭
```

```
|       exec_config → 开启, 30s 超时
```

```
|   )
```

```
|
```

```
└─ (5) 注册自定义工具 ← 关键一步
```

```
|   loop.tools.register(QueryDBTool(DB_PATH))
```

```
|   → 把 query_db 工具注入到 Agent 的工具集里
```

```
|
```

```
└─ (6) 返回 Nanobot 实例
```

```
    return Nanobot(loop)
```

build_bot: 组装整个 Agent

Thinking: 注册后, Agent 运行时可用的工具集是怎样的?

工具	来源	作用
read_file	框架内置	读文件 (加载 SKILL.md)
write_file	框架内置	写文件
exec	框架内置 (config 开启)	执行 shell 命令
query_db	自定义注册	执行只读 SQL

web_search、web_fetch 因为 config 里 web.enable: false 所以不会注册。

最后在 main() 里调用:

```
result = await bot.run(question, session_key="sql:run", hooks=[PrintHook()])
```

- question -- 用户的自然语言问题
- session_key="sql:run" -- 会话标识, 用于缓存历史到 sessions/sql_run.jsonl
- hooks=[PrintHook()] -- 挂上打印钩子, 显示工具调用过程

Summary

整个设计的核心思想是：

- 框架提供通用能力（文件读写、exec）
- 项目通过 `tools.register()` 注入领域专用工具（`query_db`）
- 通过 Skills 提供领域知识（如何探索 schema、如何写 SQL、如何纠错）
- 框架和Skill 各司其职

框架不需要懂 SQL，Skill 不需要懂框架内部实现

Config.json 设置

```
{
  "agents": {
    "defaults": {
      "workspace": ".",
      "model": "qwen-plus",
      "max_tokens": 4096,
      "context_window_tokens": 32768,
      "temperature": 0,
      "max_tool_iterations": 20,
      "timezone": "Asia/Shanghai"
    }
  },
  "providers": {
    "dashscope": { "api_key": "" }
  },
}
```

```
"tools": {
  "web": { "enable": false },
  "exec": { "enable": true, "timeout": 30 }
}
```

web: enable: false -- 关闭了网络搜索（因为查数据库不需要上网）

exec: enable: true -- exec 是开着的，但是AGENTS.md 用 prompt 约束了模型不要用 exec 来执行 SQL

AGENTS.md 设置

```
# SQL Query Agent
```

```
You are a SQL database analyst. You help users query databases using natural language.
```

```
## Rules
```

- Always explore the schema before writing queries
- Use query_db tool to execute SQL; NEVER use exec for SQL
- Only use SELECT statements (read-only access)
- Format results clearly for the user
- If a query fails, analyze the error and try a different approach

Thinking: 如何用 exec 执行 SQL是危险的吗?

看 QueryDBTool 的安全检查:

```
upper = sql.upper().lstrip()
if not (upper.startswith("SELECT") or upper.startswith("PRAGMA")):
    return "Error: only SELECT and PRAGMA statements are allowed"
```

代码层面强制只允许 SELECT 和 PRAGMA, 模型即使犯错也无法执行 DELETE、DROP、INSERT 等危险操作。

而 exec 工具是通用 shell, 如果模型用 exec 去跑 sqlite3, 就绕过了这个安全限制。

AGENTS.md

SKILL: schema-exploration

Thinking: schema-exploration 如何探索数据库结构?

思路是: 用 SQL 查 SQL -- SQLite 本身提供了元数据查询能力。

Workflow

1. Use `query\_db(sql="SELECT name FROM sqlite\_master WHERE type='table'")` to list all tables
2. For each relevant table, run `query\_db(sql="PRAGMA table\_info(TableName)")` to see columns
3. Identify primary keys, foreign keys, and relationships
4. Summarize the schema for the user

三步走:

第一步: 列出所有表

```
SELECT name FROM sqlite_master WHERE type='table'
```

sqlite_master 是 SQLite 的内置元数据表, 执行后返回



name

Album

Artist

Customer

Employee

Genre

Invoice

InvoiceLine

MediaType

Playlist

PlaylistTrack

Track

SKILL: schema-exploration

第二步：查看每张表的列信息

```
PRAGMA table_info(Customer)
```

PRAGMA table_info 是 SQLite 的内置命令，返回表的列定义：

cid	name	type	notnull	dflt_value	pk
0	CustomerId	INTEGER	1	None	1 <-- 主键
1	FirstName	NVARCHAR(40)	1	None	0
2	LastName	NVARCHAR(20)	1	None	0
3	Company	NVARCHAR(80)	0	None	0
4	Country	NVARCHAR(40)	0	None	0
...					



SKILL: schema-exploration

第三步：分析关系

模型通过列名推断外键关系，比如看到 `Invoice.CustomerId` 就知道 `Invoice` 表通过 `CustomerId` 关联 `Customer` 表。

整个流程图：

用户: "How many customers are from Canada?"

=> 模型加载 schema-exploration SKILL.md (渐进式/always)

=> `query_db("SELECT name FROM sqlite_master WHERE type='table'")`

└ 知道了有 `Customer` 表

=> `query_db("PRAGMA table_info(Customer)")`

└ 知道了有 `Country` 列

=> 模型加载 query-writing SKILL.md

=> `query_db("SELECT COUNT(*) FROM Customer WHERE Country = 'Canada'")`

└ 结果: 8

└ 回答: "There are 8 customers from Canada."

SKILL: query-writing

query-writing SKILL.md 完整结构:

name: query-writing

description: "Write and execute SQL queries to answer user questions about the database"

Query Writing Skill

When to Use

- User asks a question that requires querying the database
- User wants to analyze, count, aggregate, or compare data

Workflow

Step 1: Understand the Question

- Identify what tables and columns are needed
- If unsure about schema, read the `schema-exploration` skill first

Step 2: Write the Query

- Start simple, then add complexity
- Use JOINS when data spans multiple tables

- Use GROUP BY for aggregations

- Use ORDER BY + LIMIT for "top N" questions

- Always use table aliases for clarity

Step 3: Execute and Verify

- Run with `query\_db(sql="YOUR QUERY")`
- If error: analyze the message, fix the query, retry
- If results look wrong: check JOINS and WHERE clauses

Step 4: Present Results

- Format results as a clear answer in natural language
- Include the actual numbers/data
- For complex results, use a table format

Error Recovery

- "no such table" -> re-list tables with schema-exploration
- "no such column" -> re-check table_info
- "ambiguous column" -> add table alias prefix

query-writing/SKILL.md

运行结果

运行命令: `python agent.py "How many customers are from Canada?"`

Text-to-SQL Agent (nanobot)

Question: How many customers are from Canada?

Answer: There are 8 customers from Canada in the database.

exit_code: 0 | elapsed: 35s

如果上一次运行的 session 已缓存 schema 信息 (sessions/sql_run.jsonl) , 那么Agent 本次直接命中正确查询, 无需重新探索 schema。

首次运行时 Agent 会先查询 customers(小写) 失败, 然后自动用 sqlite_master 列出所有表名、用 PRAGMA table_info 检查列, 修正为 Customer(大写) 重新查询。

=> 体现了 Skills 中 "Error Recovery" 指导的效果。

CASE: Deep Research (nanobot)

CASE: Deep Research (nanobot)

TO DO: 搭建 Deep Research 深度研究

用户输入一个研究问题，Agent 自动联网搜索、反思、整合，最终输出一篇带引用的研究报告。

项目结构：

```
nanobot-examples/deep-research/  
  config.json -- qwen-plus + tavily  
  AGENTS.md -- Research methodology 四步法  
  agent.py -- 入口 + ThinkTool 自定义工具  
  memory/ -- 当前日期上下文
```

自定义 ThinkTool 作为反思机制，让 Agent 在搜索后评估信息完整度，决定是否继续搜索。

CASE: Deep Research (nanobot)

运行命令: `python agent.py "List 2 trends in AI agents in one paragraph"`

Deep Research Agent (nanobot)

Question: List 2 trends in AI agents in one paragraph

Agent Output:

Two prominent trends in AI agents are the rise of agentic workflows, where agents autonomously plan, execute, and reflect across multi-step tasks -- leveraging tools, memory, and dynamic reasoning -- and the increasing adoption of lightweight, domain-specialized agents, often designed for local or edge deployment, emphasizing speed, transparency, and data sovereignty over broad generality. These shifts signal a move from static chat interfaces toward adaptive, tool-integrated systems that operate reliably within specific contexts -- such as finance, healthcare, or developer tooling.

exit_code: 0 | elapsed: 45s

CASE: Deep Research (nanobot)

Thinking: ThinkTool 的作用是什么？

一个空操作工具，execute 方法只是把模型的反思内容原样返回，不做任何实际计算。

=> 强迫模型在搜索之间停下来结构化思考，而不是一口气搜完就写结论。

模型调用 think_tool(reflection="...") 时，思考内容会作为工具结果记录在对话历史中，成为后续推理的上下文。

本质上是用工具调用的形式实现了思维链。

CASE: Deep Research (nanobot)

执行逻辑 (对应 AGENTS.md 的四步法) :

用户问题 → Plan (拆分子主题)

→ Search (web_search 搜索每个子主题)

→ Reflect (think_tool 反思: 找到了什么? 还缺什么? 要不要继续搜?)

→ 搜索不够 → 回到 Search 继续

→ 搜索够了 → Synthesize (write_file 写 final_report.md)

Summary

Nanobot vs DeepAgents 实际运行对比

指标	DeepAgents (Qwen 适配版)	Nanobot 版
Content Builder 耗时	161s	39s (快 4 倍)
Text-to-SQL 耗时	163s	29s (快 5.6 倍)
Deep Research 耗时	271s	24s (快 11 倍)
Windows 编码问题	需手动修复 UnicodeEncodeError	无 (框架内部已处理)
模型切换复杂度	修改 Python 代码 + 导入语句	改 config.json 一行
代码量 (入口脚本)	300+ 行	80-120 行

Thinking: 为什么 Nanobot 版更快?

DeepAgents 版的 content-builder/deep_research 使用了子 Agent 机制 (researcher/writer 分工) , 增加了多次 LLM 调用开销。

Nanobot 版让单个 Agent 直接完成所有步骤, 减少了子 Agent 通信延迟。

两种架构各有优劣: 子 Agent 模式在复杂任务中分工更细 => 适合长篇内容生成。

CASE: nanobot下单

CASE: nanobot下单

Thinking: AI自己发现了一个好策略, 自动下单? 还是进行询问, 需要人的授权?

AI 建议, 人类授权

用户提问 --> Agent 分析 --> AI 给出建议 --> 人类授权 --> 执行下单 --> 反馈结果

Thinking: AI 建议要包含什么?

- 策略依据, 比如: MACD 金叉, 历史胜率 55%
- 当前持仓, 比如: 当前无 513100 持仓
- 资金情况, 比如: 可用资金 3 万
- 建议操作, 比如: 建议市价买入 200 股 (约 xx 元)

场景1：查看策略信号

场景1：查看策略信号

输入：帮我看下 513100 的 MACD 信号，最近有没有金叉

```
exec: run_backtest.py --code 513100.SH --strategy macd --count 250
```

513100 纳指ETF MACD信号分析 (截至2026-04-10)

投资建议: 当前MACD显示多头趋势, 但历史胜率较低 (28.6%) , 需谨慎。建议持续监控, 等待真正的金叉出现, 结合其他指标综合判断。

最新信号: bullish (多头趋势)

最新收盘价: 1.809元

当前状态: DIF线位于DEA线上方, 但最近未发生金叉

策略绩效统计:

- 过去250个交易日共产生 14次 交易信号
- 胜率: 28.6% (4次盈利/14次交易)
- 总收益率: -10.08%
- 最大回撤: 15.32%

场景2：策略回测验证

场景2：策略回测验证

输入：回测一下双均线策略在 513100 上的表现

exec: run_backtest.py --code 513100.SH --strategy double_ma --count 250

513100 双均线策略回测报告 (MA5/MA20)

回测周期: 2025-04-28 ~ 2026-04-10 (231个交易日)

绩效统计: 交易7次 | 胜率28.6% | 总收益 +7.15% | 最大回撤 5.55%

近期交易记录:

买入日期	卖出日期	买入价	卖出价	收益率
2025-06-24	2025-08-25	1.595	1.698	+6.46%
2025-09-10	2025-11-18	1.711	1.844	+7.77%
2025-12-01	2025-12-16	1.936	1.873	-3.25%
2026-01-07	2026-01-08	1.919	1.899	-1.04%

与MACD策略对比: 双均线 +7.15% vs MACD -10.08%, 双均线回撤更小 (5.55% vs 15.32%), 交易更少(7次 vs 14次)。

建议: 双均线在513100上表现优于MACD。当前处于多头趋势, 但价格已从高点回调约6.5%。

场景3：下单请求

场景3：下单请求

输入：买100股 513100

```
exec: query_account.py --action asset
```

```
exec: query_account.py --action positions
```

```
exec: query_account.py --action orders
```

账户现状分析:

- 总资产: 0.0元 | 可用资金: 0.0元
- 已持有 513100.SH: 100股 (成本1.926元)
- 已持有 002432.SZ: 3000股
- 发现6笔未成交委托 (价格0.5~1.809元)

重要发现: 账户可用资金为0元!

无法执行买入操作。建议:

1. 先充值资金到交易账户
2. 或撤销现有未成交的无效委托 (0.5元的委托明显不合理)

3. 然后重新下单

需要我帮您撤销所有未成交的委托吗?

用户说买

=> Agent 没有盲目执行，而是先查询了资产、持仓、委托

=> 发现资金不足后主动拒绝并给出了清晰的替代建议。

=> AI 帮你分析，而非替你决定。

场景4：查看持仓和委托

场景4：查看持仓和委托

输入：查看我的持仓和今日委托

当前持仓（2只股票）：

股票代码	名称	持仓	可用	成本价	市值
513100.SH	纳指ETF	100股	100股	1.926元	180.90元
002432.SZ	九安医疗	3000股	3000股	49.517元	278,250元

总持仓市值: 278,430.90元

今日委托（6笔未成交）：

订单ID	委托价	数量	状态	问题
672137217	0.500元	100股	未成交	价格远低于市价
672137218	1.986元	100股	未成交	高于市价
672137219	1.809元	100股	未成交	等于市价
672137220	0.500元	100股	未成交	价格过低无效

建议: 撤销所有0.500元的无效委托（4笔），评估剩余2笔合理委托。

直接引用上轮查询结果，无需重复调用

场景5：研报分析 + 交易决策

场景5：研报分析 + 交易决策

输入：分析下 513100 近期走势，看看能不能买

stock-price 获取K线

--> strategy-backtest 回测信号

--> trade-order 查持仓/资金

--> AI 建议方案 --> 人类授权

--> trade-order 执行下单

Agent 协调多个 Skill，从数据获取到策略分析到交易执行，作为决策中枢串联整个流程。

Nanobot SDK编程接口

Nanobot SDK 编程接口

最简用法

```
import asyncio
from nanobot.nanobot import Nanobot

async def main():
    bot = Nanobot.from_config("config.json", workspace=".")
    result = await bot.run("Hello, world!")
    print(result.content)

asyncio.run(main())
```

带 Hook 的高级用法

```
from nanobot.agent.hook import AgentHook,
AgentHookContext

class MyHook(AgentHook):
    async def before_execute_tools(self, ctx:
AgentHookContext):
        for tc in ctx.tool_calls:
            print(f"Calling: {tc.name}({tc.arguments})")

result = await bot.run("Analyze this", hooks=[MyHook()])
```

Nanobot SDK 编程接口

RunResult 返回值

```
@dataclass
class RunResult:
    content: str      # Agent 的文本回复
    tools_used: list[str] # 使用过的工具名列表
    messages: list[dict] # 完整对话历史
```

Summary (架构选型)

- 快速构建领域 Agent

Nanobot => 配置驱动, Skills 开箱即用

- 复杂多 Agent 编排

DeepAgents => LangGraph 状态机 + 丰富的中间件

- 需要切换不同 LLM

Nanobot => 改 config.json 一行, 无需改代码

- 需要精细控制 Agent 行为

DeepAgents => 中间件可拦截/修改每次请求和响应

Summary (Skills 设计原则)

- MECE 原则:

每个 SKILL.md 负责一个独立的能力域，互不重叠。

- 自包含:

每个 SKILL.md 包含完整的使用指南（适用场景、脚本命令、示例对话），Agent 读取后即可执行。

- 索引技能:

创建一个 "meta skill"（如 investment-research）作为技能索引，帮助 Agent 快速定位应该使用哪个技能。

- description 精准:

用一句话概括技能用途，包含关键触发词（如"联网搜索"、"PDF 财报"、"K 线行情"）。

Summary

Thinking: 如何从 DeepAgents 迁移到 Nanobot?

- 创建 config.json: 配置模型、提供商、工具开关
- 编写 AGENTS.md: 将原 system_prompt 中的身份描述提取出来
- 拆分 Skills: 将原 system_prompt 中的能力指导拆分为多个 SKILL.md
- 迁移工具: @tool 函数 -> Tool 类 (或直接用 exec 调用脚本)
- 编写入口脚本: 使用 Nanobot SDK 构建和运行 Agent
- 验证: 检查 Agent 是否能正确加载 Skills 和调用工具

CASE: 打造 Skill Creator

CASE: 打造 Skill Creator

Thinking: 业务人员创建 Skill 的痛点有哪些?

业务人员有丰富的工作经验, 但让他们直接写成 Agent 能执行的 Skill 时, 通常面临以下问题:

业务人员自己写的 Skill (典型问题)

"帮我做个分析" -- 太泛, 分析什么? 看什么指标?

"查一下数据" -- 哪个系统? 什么字段? 怎么判断好坏?

"写个结论" -- 结论包含哪几部分? 格式是什么?

步骤跳跃 -- 从 A 直接到 D, 中间 B、C 省略了 (因为对本人来说是"常识")

缺少边界 -- 没有说什么不能做、什么必须做

经过 biz-skill-creator 引导后的 Skill

场景清晰: "为基金经理团队生成8:30微信群简报"

步骤具体: "从Wind拉取美股指数 -> 计算涨跌幅 -> 格式化为+X.X%"

输出明确: 有模板、有占位符、有示例

规则完整: "必须含判断性短语" "不使用markdown"

可测试: examples.md 提供了验证用的参考案例

业务人员不是不懂, 而是不知道该说什么、说到什么粒度。

=> biz-skill-creator 的价值不是 帮他写, 而是 帮他问出来。

biz-skill-creator 的三层架构设计

biz-skill-creator 本质上是一个结构化访谈系统。由 3 个文件组成，每个文件解决一个层面的问题：

biz-skill-creator/

SKILL.md [第1层: 流程控制] "问什么 + 什么时候问下一步"

[references/](#)

interview-guide.md [第2层: 追问能力] "用户说了模糊的话怎么追问"

skill-template.md [第3层: 输出约束] "最终生成物长什么样"

第一层：SKILL.md（流程控制）

第一层：SKILL.md -- 流程控制（大脑）

定义 4 步固定流程，控制对话节奏。每步有明确的提问内容和完成标准。

Step1, 场景定位

做什么？什么时候做？输入/输出？

Step2, 示例采集

真实案例，具体步骤，示例输出

Step3, 规则提炼

必须做的事，禁止做的事，参考文档

Step4, 生成确认

套模版生成，用户审阅，修改迭代

第一层：SKILL.md（流程控制）

步骤	类比	解决的问题	完成标准
Step 1 场景定位	记者问 5W1H	业务人员说不清"到底要做什么"	能用一段话描述: 在什么场景下, 拿到什么输入, 产出什么结果
Step 2 示例采集	观察师傅操作一遍	抽象描述不如具体案例	至少 1 个完整案例: 输入 + 步骤序列 + 输出
Step 3 规则提炼	问师傅"有什么忌讳"	隐性知识最难表达	至少 1 条"必须做" + 1 条"不能做"
Step 4 生成确认	写成操作手册让师傅过目	结构化输出 + 人工确认	用户确认 SKILL.md 内容准确

流程是固定的，不是开放式提问 => 避免了 Agent 随意发散或遗漏关键信息。

每步都有完成标准，Agent 必须达标才能推进到下一步。

第二层：interview-guide.md （追问能力）

第二层：interview-guide.md -- 追问能力（"话术手册"）

这是 biz-skill-creator 最关键的差异化设计。

没有这一层，Agent 遇到模糊回答只会接受并继续，生成出来的 Skill 就会很泛。

核心机制: 追问策略表 -- 针对每种常见的模糊回答，预设了具体的追问话术:

用户说了什么（模糊回答）	Agent 怎么追问
"我会分析一下"	"分析具体看哪些指标？怎么判断好还是不好？"
"查一下数据"	"从哪个系统查？查哪些字段或指标？"
"对比一下"	"跟什么对比？上一期？同行？对比哪些维度？"
"写个结论"	"结论一般包含哪几部分？长度大概多少？能给个例子吗？"
"看情况而定"	"能举两种不同情况的例子吗？分别是怎么处理的？"
步骤之间跳跃太大	"在 xx 和 yy 之间，有没有其他中间步骤？"
"没什么特别的"（规则环节）	"我注意到你在案例中提到了 xx，这是不是每次都必须做的？"
规则太多太细	"如果只能保留最重要的 3 条，你会选哪 3 条？"

第二层：interview-guide.md（追问能力）

Thinking：为什么需要单独文件？

如果把追问策略直接写在 SKILL.md 里，会导致 SKILL.md 太长（超过 200 行），增加 LLM 遗漏关键信息的风险。

分离到 interview-guide.md 后，Agent 在需要时通过 read_file 按需加载。

=> nanobot 的渐进式加载（progressive disclosure）设计理念。

第二层：interview-guide.md（追问能力）

引导问题话术库

业务人员通常不善于结构化表达。本文档提供每一步的标准话术和追问策略。

Step 1: 场景定位 -- 话术

开场白

"接下来我会通过几个简单的问题，帮你把这个经验整理成一个可复用的技能。每步只需要回答 2-3 个问题，我们开始吧。"

标准提问

"请回答以下 3 个问题：

1. ****你在做什么？**** 用一句话描述这个任务（比如：帮客户评估某只债券的信用风险）
2. ****什么情况下会做这件事？**** 列 1-3 个触发场景（比如：客户主动问、新债上市时、评级变动时）
3. ****输入和输出是什么？**** 你拿到什么信息，最终产出什么？（比如：拿到公司名称和债券代码，产出一段信用评估结论）"

追问策略

| 用户回答 | 追问话术 |

|-----|-----|

| 太宽泛，如"帮客户做分析" | "做什么类型的分析？能举一个最常见的例子吗？" |

| 只说了做什么，没说什么时候 | "一般什么情况下你会做这件事？是客户来问，还是定期做，还是有什么事件触发？" |

| 没说清输入输出 | "做这件事的时候，你手上通常已经有什么信息？最终要交出去的是什么的形式的东西？" |

Step 2: 示例采集 -- 话术

标准提问

"现在请给我讲一个你实际做过的案例，越具体越好：

1. 当时的具体输入是什么？（比如：拿到了 xx 公司 2025 年年报）
2. 你做了哪些步骤？一步一步说
3. 每一步参考了什么资料或系统？
4. 最终输出是什么样的？能给一段示例内容吗？"

第二层：interview-guide.md（追问能力）

追问策略

这是最容易出现"太简略"的环节，需要重点追问。

| 用户回答 | 追问话术 |

|-----|-----|

| "我会分析一下" | "分析的时候具体看哪些指标或内容？怎么判断好还是不好？" |

| "查一下数据" | "从哪个系统查？查哪些字段或指标？" |

| "对比一下" | "跟什么对比？上一期？同行？对比哪些维度？" |

| "写个结论" | "结论一般包含哪几部分？长度大概多少？能给一个例子吗？" |

| "看情况而定" | "能举两种不同情况的例子吗？分别是怎么处理的？" |

| 步骤之间跳跃太大 | "在 XX 和 YY 之间，有没有其他中间步骤？比如需要验证什么，或者转换什么格式？" |

| 只说了一个案例且很简单 | "还有没有一个稍微复杂一点的案例？或者遇到过什么特殊情况？" |

示例引导

如果用户完全不知道怎么说，可以给一个完整示例作为参考：

"比如，一个'债券信用评估'的案例可能是这样的：

输入：客户问'xx 城投债能不能买'

步骤：

1. 先查 xx 公司的最新信用评级（中诚信、联合资信等）
2. 下载最近一期年报，看资产负债率、经营性现金流
3. 查看该地区的财政收入和债务率数据
4. 和同评级同地区的城投做横向对比
5. 综合判断，写出评估结论

输出：'建议关注/建议规避 + 理由 + 主要风险点'

你的案例也可以按这个格式来说。"

Step 3: 规则提炼 -- 话术

标准提问

"最后几个问题，帮我提炼一下关键规则：

第二层：interview-guide.md（追问能力）

1. ****必须做的事****: 做这个任务有什么不能跳过的步骤或者必须遵守的原则？
2. ****绝对不能做的事****: 有什么常见的错误或者禁区？做了会出大问题的？
3. ****参考文档****: 做这个任务时你会参考哪些文件、手册、系统或数据源？ "

追问策略

| 用户回答 | 追问话术 |

|-----|-----|

| "没什么特别的" | "我注意到你在案例中提到了 xx（从 Step 2 提取），这不是每次都必须做的？" |

| 只说了"必须"没说"不能" | "有没有什么常见的误区？新人刚开始做的时候容易犯什么错？" |

| 参考文档为空 | "做的时候完全凭经验？不需要查任何系统或文件？" |

| 规则太多太细 | "如果只能保留最重要的 3 条，你会选哪 3 条？" |

Step 4: 生成确认 -- 话术

展示成果

"好的，我已经根据你的描述生成了技能文件。请看一下以下内容是否准确：

（展示生成的 SKILL.md 内容）

你觉得有什么需要修改的吗？比如：

- 步骤有遗漏或顺序不对？
- 规则需要补充或修改？
- 描述不够准确？ "

确认完成

"技能已经创建完成，保存在 skills/{skill-name}/ 目录下。以后 Agent 遇到相关任务时就会自动参考这个技能来执行了。"

第三层： skill-template.md （输出约束）

第三层： skill-template.md -- 输出约束（模具）

定义生成的 SKILL.md 和 examples.md 必须遵循的固定格式。确保不同人创建的技能结构一致、可维护。

SKILL.md 标准模板结构:

```
---
name: {小写字母+连字符}
description: "{做什么}。当{触发场景}时使用。"
---
# {技能标题}
## 操作流程      ← 从 Step 2 示例中提炼，祈使句
## 关键规则
### 必须遵守      ← 从 Step 3 "必须做的事" 映射
### 禁止事项      ← 从 Step 3 "不能做的事" 映射
## 参考文档      ← 从 Step 3 "参考文档" 映射
```

字段	填写规则	为什么重要
description	必须同时包含 WHAT（做什么）+ WHEN（什么时候）	Agent 靠 description 判断何时触发该技能；缺少 WHEN 会导致误触发
操作流程	祈使句，编号，每步一行	Agent 按顺序执行；模糊的步骤会导致 Agent 自由发挥
必须遵守 / 禁止事项	分开列出，一句话说清	没有"禁止事项"，Agent 可能犯业务上不可接受的错误

四个设计原则

1. 固定流程 > 开放提问

不问“你想创建什么样的技能？”，而是严格按 4 步走。

原因: 开放式提问让业务人员不知道从哪说起，固定流程降低认知负担。

2. 示例 > 抽象描述

Step 2（示例采集）是整个流程的重心。

原因: 业务人员说“我会分析”时自己也说不清具体怎么分析，但让他讲一个真实案例时，步骤自然就出来了。

SKILL.md 中明确写了：“示例比抽象描述更有价值，优先采集示例”。

四个设计原则

3. 每步只问 2-3 个问题

一次抛出 10 个问题，用户会放弃。

每步控制在 2-3 个问题，降低心理压力。用户回答后再追问，逐步深入。

4. 业务语言 > 技术术语

整个过程中不出现 frontmatter、token、context window 等技术词汇。

用操作步骤代替 workflow，用"做什么 + 什么时候做"代替 description 字段。

Skill Creator创建过程

我想创建一个技能，是每日晨会简报的

进入 Step 1: 场景定位。

主动追问: 给谁看？每天几点？手动整理哪些信息？最希望 AI 完成什么？ 给出引导示例。

给基金经理团队看的，每天8:30发到群里。手动整理隔夜美股、A股行情、经济数据、个股公告。希望AI自动汇总。

提炼出场景定位总结。

主动追问扩展方向: 是否需要港股/期货？关联持仓列表？表格 or bullet point？

Skill Creator创建过程

不需要港股期货。bullet point。不关联持仓。最后加策略观点占位符，人工写的。

确认修正，过渡到 Step 2: 示例采集。

要求提供手写简报样本，给出风格对比示例。

(完整的 4 月 7 日晨会简报: 隔夜市场回顾 + 今日重点关注 + A股盘前要点 + 策略提示)

逐模块解析示例特征。直接跳入 Step 3, 自动提炼 5 条规则 (时间逻辑、数据来源、表达规范、风险标亮、容错)。

主动建议改名"A股盘前要点" → "A股昨夜今晨要点"。

Skill Creator创建过程

规则大部分OK。5 个调整:

- 1)名称不改
- 2)不标来源
- 3)不用加粗
- 4)不算连续天数
- 5)可以生成了。

进入 Step 4: 生成确认。

工具链: `list_dir` → `exec(mkdir)` → `write_file(SKILL.md)` → `write_file(examples.md)`

生成质量评估: daily-morning-brief

- 采集质量 (9/10)

Agent 主动追问了港股/格式/持仓等细节，从示例中自动提炼了 5 条规则，采纳了用户的 5 个调整意见

- 追问能力 (8/10)

追问了 3 个扩展方向（港股/持仓/格式偏好），还主动建议改名"A股昨夜今晨要点"（虽然被用户否决，但体现了独立判断能力）

- 模板遵循 (4/10)

没有遵循 skill-template.md 的标准结构（操作流程/关键规则/参考文档），而是自创了章节（使用场景/触发条件/输入/输出/执行流程）

生成质量评估: daily-morning-brief

- 规则落地 (5/10)

对话中提炼的 5 条规则 (+X.X%格式、判断性短语、不用markdown等) 被散落到各处, 未集中在关键规则章节

- 示例结构 (4/10)

examples.md 是用户原始简报的照搬, 未按模板分为"输入/执行步骤/输出"三段结构

生成质量评估: daily-morning-brief

具体问题分析

问题	期望行为	实际行为	严重度
SKILL.md 结构偏离模板	按 skill-template.md 生成: 操作流程 / 关键规则(必须+禁止) / 参考文档	自创了: 使用场景 / 触发条件 / 输入 / 输出 / 执行流程 / 示例	高
Step 3 规则未结构化落地	将"必须+X.X%格式""禁止markdown"等规则写入"关键规则"章节	规则被散落在"输出"和"执行流程"描述中, 无独立的"关键规则"章节	高
examples.md 未结构化	分为 输入 / 执行步骤 / 输出 三段	原文照搬用户手写简报	中
YAML frontmatter 格式错误	有开头和结尾的 ---	缺少结尾的 --- (已手动修复)	中
无"参考文档"章节	列出 Wind 终端等数据源	完全缺失	低

采集环节 (Step 1-3) 表现优秀 (8-8.5 分) , Agent 能有效引导业务用户。

但生成环节 (Step 4) 是短板 (4 分)

已知限制

根因: qwen-plus 的渐进式 Skill 加载不稳定

nanobot 的 Skill 系统采用渐进式加载 (progressive disclosure) 设计:

1. Agent 启动时, 只加载每个 Skill 的 name 和 description (几行文字)
2. 当 Agent 判断需要某个 Skill 时, 通过 read_file 读取完整的 SKILL.md
3. Skill 中引用的 references/ 文件, 需要 Agent 再次 read_file 加载

问题出在第 3 步: Step 4 的 SKILL.md 里写了"读取 references/skill-template.md 获取输出模板", 但 qwen-plus 有时不会执行这个 read_file, 而是凭已有上下文中的记忆自由生成。这导致:

- 生成的 SKILL.md 结构偏离模板 (最常见的问题)
- examples.md 未按模板格式化
- 有时语言要求也被忽略 (如之前出现的英文输出)

已知限制

Thinking: 有什么解决办法? 可以让qwen-plus读取到 SKILL.md 的完整内容?

这是模型层面的限制, 非 biz-skill-creator 设计问题。

qwen-plus 对多层级的 read_file 调用 (Skill 引用 reference, reference 中再引用其他文件) 支持不够稳定。
在 Claude 或 GPT-5 等模型上, 这个问题会显著减轻。

对于 qwen-plus, 可考虑将模板内容直接内联到 SKILL.md 中

=> 牺牲渐进式加载, 换取输出稳定性

使用 nanobot 的 always: true 标记强制加载。

Summary

- 采集环节评价:

4 步流程运行顺畅。Agent 能主动追问细节（港股/格式/持仓），
从示例中自动提炼规则，采纳用户的调整意见，并展现独立判断能力（建议改名）。

- 生成环节评价:

输出内容有用但结构偏离了 skill-template.md。

Agent 未调用 read_file(skill-template.md) 就直接生成，导致缺少"关键规则""参考文档"等标准章节。

=> 这是 qwen-plus 渐进式加载不稳定的已知限制。

Summary

- biz-skill-creator

引导业务人员将个人经验沉淀为 Agent Skill。是整个项目的元技能 -- 用来创建其他技能。

文件	作用	行数
SKILL.md	第 1 层: 4 步流程控制 + 核心原则 + 生成约束 + 中文语言要求	94
references/interview-guide.md	第 2 层: 每步标准话术 + 8 类追问策略 + 完整示例引导	105
references/skill-template.md	第 3 层: SKILL.md 模板 + examples.md 模板 + 字段填写规范	106

Summary

- web-search

联网搜索实时市场信息。通过 exec 调用 search_market.py，底层使用 qwen 的 enable_search 能力。

搜索类型	用途	示例
stock	个股行情	"贵州茅台最新股价行情"
news	财经新闻	"半导体行业最新新闻"
policy	政策法规	"半导体行业最新政策利好"
general	通用搜索	"中芯国际 券商评级 最新"

Summary

- 技术配置

参数	值	说明
model	qwen-plus	通义千问-plus
context_window_tokens	65536	上下文窗口 64K
temperature	0.1	低温度保证输出稳定
max_tool_iterations	30	最多 30 轮工具调用
exec.enable	true	允许执行脚本 (web-search 依赖)

Summary

技能全景图

- stock-price: K线行情、最新价格
- strategy-backtest: MACD/双均线回测、信号检测
- trade-order: 买入/卖出/撤单/查询
- write-report: 研报生成并导出 HTML
- read-pdf: 研报/财报 PDF RAG
- financial-analysis: 财务指标分析
- sentiment-analysis: 舆情监控、情感分析
- compare-reports: 跨期/跨公司对比

Summary

Agent + Skills 的优势:

- 技能按需加载, 不浪费 context window
- 每个技能自包含, 可独立开发测试
- Agent 自主决定调用顺序和组合方式
- 人类始终保持对关键操作 (交易) 的控制权



Thank You
Using data to solve problems

